

GAZBOT

Product & Architecture Dossier — prepared 18 August 2026

CONFIDENTIAL Prepared for commercial product planning. Describes architecture and capability only. **No trading logic, entry/exit conditions, thresholds or parameter values are disclosed in this document.**

Document set

1. **Product & Architecture Dossier** — how the machine works (this document, Parts I–XV)
2. **Capability Inventory** — capability → what it does → why it matters → customer benefit → maturity (Part XVI)
3. **The Skeptic's Ten** — what a professional systematic trader would find genuinely differentiated (Part XVII)

I. Executive overview

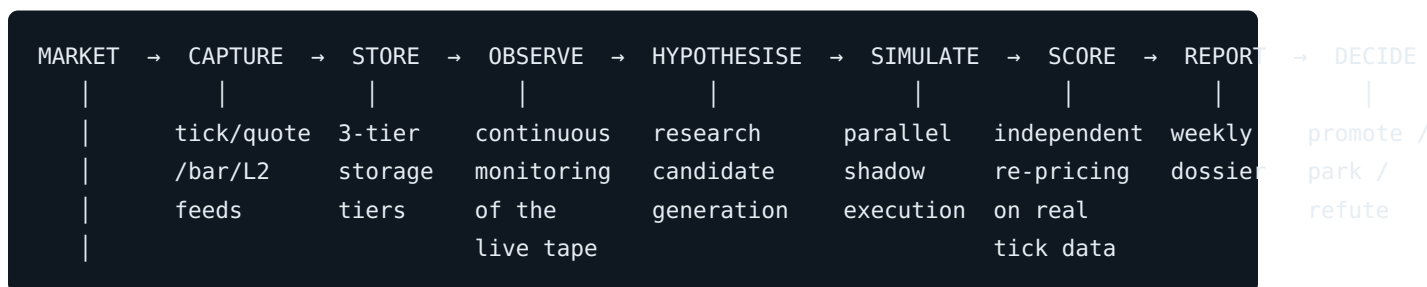
GAZBOT is a **continuously-running quantitative research machine** that happens to also trade. Its defining property is not that it places orders — thousands of products do that — but that it **accumulates research**: every market observation, every simulated decision, every experiment result and every refuted idea is written to a queryable historical store and remains available to all future research, indefinitely.

Repositioned as a research agent rather than an execution bot, the product is best described as: an always-on instrument that watches a market, forms and tests hypotheses about it, records the verdicts permanently, and produces a weekly scientific report on what it learned.

Scale of the working system, as measured

Dimension	Current
Application code	~16,100 lines across 30 core modules, plus 308 analysis/operations scripts
Automated tests	82 test modules, ~830 tests, gating every change
Orchestrated processes	63 systemd units — continuous services, scheduled jobs and one-shot timers
Market data held	55.1M order-book rows, 11.2M L2 depth snapshots, 6.5M quotes, 5.3M trade ticks, 567k bars
Storage footprint	5.3 GB hot tape, 2.8 GB L2 depth, 702 MB Parquet lake (163 partitions), plus off-site object storage
Research records	10,807 simulated decisions, 100% independently re-priced and scored
Registered experiments	82, each with a permanent immutable identifier
Report pipeline	21-phase weekly research report, fully automated

II. What GAZBOT does, end to end



1. **Capture.** Broker market data is consumed continuously and written to local storage — trade ticks, quotes, aggregated bars, top-of-book and full 10-level depth snapshots at sub-second cadence.
2. **Store.** Data lands in a hot transactional tier, is mirrored nightly to a columnar analytical lake, and is replicated off-site. Each tier has a different lifetime and a different job.
3. **Observe.** A live decision layer reads the tape continuously and maintains a structured, machine-readable view of market state.
4. **Hypothesise.** Candidate research ideas are generated from observed structure, from failure analysis of prior decisions, and from the weekly report's own findings.
5. **Simulate.** Candidates run as shadow arms — full decision logic executing against the live tape in parallel with the real desk, placing no orders. Dozens run concurrently.
6. **Score.** Every simulated decision is independently re-priced against captured tick data, producing an honest fill-aware result rather than an optimistic bar-price approximation.

7. **Report.** A 21-phase weekly pipeline analyses the accumulated record and produces a structured research dossier with findings, verdicts and proposed next experiments.
8. **Decide.** Findings are promoted, parked or refuted — and the verdict itself is stored, so the same ground is never re-litigated from scratch.

III. What problem it solves

The central problem in systematic trading research is **not** generating ideas. It is that research evaporates. A typical practitioner:

- runs a backtest in a notebook, gets a number, and loses the notebook;
- cannot reconstruct what data, code version or assumptions produced last month's result;
- re-tests ideas that were already refuted, because the refutation was a conversation and not a record;
- has no measurement of what a rejected idea would have done since rejection;
- discovers too late that a promising backtest was an artefact of a data or methodology defect.

GAZBOT's answer is to treat research as **durable infrastructure** rather than as activity. Ideas become registered, numbered, permanently-identified experiments. Results become rows. Verdicts become records with the evidence attached. A refuted idea keeps running in simulation so the refutation stays live rather than becoming folklore.

IV. What makes it different from conventional trading bots

Dimension	Conventional bot / strategy platform	GAZBOT
Primary artefact	Orders	Research records
Backtest	Run on demand, discarded	Continuous forward simulation, permanently stored
Fill assumption	Bar close / mid price	Independently re-priced on captured tick data
Rejected ideas	Deleted	Kept running in shadow so the rejection is continuously tested
Controls	None	Deliberate control arms that are expected to lose, as the baseline
Memory	None between sessions	Permanent registry, session log, decision ledger, architectural record
Self-monitoring	"Is the process alive?"	Instrumented against the specific failure of reporting healthy about something it never checked
Reporting	Equity curve	Multi-phase weekly research dossier with adversarial review

The distinguishing design principle. A large fraction of this system exists to prevent one specific failure: an instrument that reports healthy about a thing it does not actually check. That principle is not marketing language — it is implemented as concrete checks throughout the codebase, and several of them were built after that exact failure occurred and was diagnosed. This is a system that has been debugged by contact with reality, and it carries the scar tissue in code.

V. System architecture

Component families

Component	Role	Lifecycle
Market-data service	Single broker connection; normalises and republishes all instrument data on an internal bus	Continuous
Depth-capture service	Independent 10-level order-book snapshot recorder at sub-second cadence	Continuous
Execution desk	The live decision-and-order layer; multi-slot, multi-strategy, dual-lot position management	Continuous
Secondary desk	A structurally independent second strategy with its own broker session and its own state	Scheduled tick
Shadow research desks	Parallel simulation of the full candidate slate against the live tape; places no orders	Continuous (one per instrument)
Re-pricing engine	Independently re-scores every simulated decision against captured tick data	Interval, in-process
Reconciliation service	Enforces that broker position equals the sum of every desk's claim	Every 30 s
Router	Decides which strategy families are permitted to open new positions given current regime	Every 5 min
Web/dashboard service	Read-only presentation: desk, research board, reports, analytics cube	Continuous
Notification service	Operator alerting with situation-keyed de-duplication	Continuous
Report pipeline	21-phase weekly research dossier generator	Weekly, one process per phase
Maintenance jobs	Backup, mirror, prune, vacuum, audit, health, supervisor	Scheduled

How components communicate

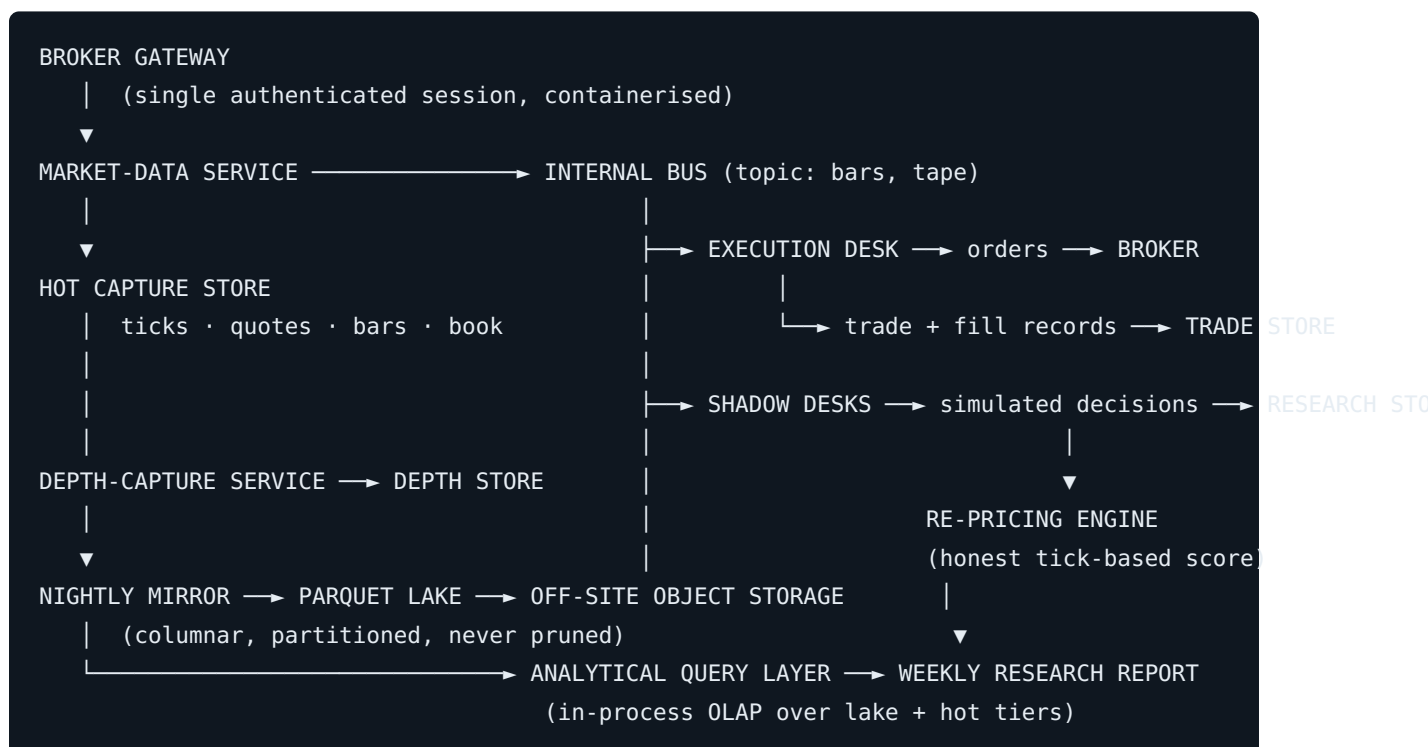
- **Internal message bus.** The market-data service publishes normalised messages on a local topic-based stream. Consumers subscribe by topic. The bus is multi-instrument, and every consumer is required to filter by instrument — a rule enforced because violating it once corrupted every downstream volatility calculation.
- **Shared files as contracts.** Cross-process state uses atomically-replaced JSON documents (health, position, switch state, open-simulation snapshots). This makes state inspectable by any process, any script, and any human, without a service dependency.

- **Databases as the integration point.** Research and analytics components integrate through the stores rather than through APIs, which means any new tool can read the entire research history with no coordination.
- **Process isolation by design.** Each desk holds its own broker session identity. A fault in one cannot cancel another's orders — this is enforced in code and verified by the reconciliation layer.

Continuous vs on-demand

Always running	Scheduled	On demand
Market data · depth capture · execution desk · shadow desks · web · notifications · event watcher	Router (5 min) · reconciliation (30 s) · secondary desk · health checks · nightly audit/backup/mirror/prune · weekly report	Research scripts · backtests · walk-forward sweeps · data inventory · ad-hoc analysis

End-to-end data flow



VI. Data architecture

The three-tier model

History lives in three places with deliberately different lifetimes. This is the single most commercially reusable piece of the system.

Tier	Technology	Lifetime	Purpose
Hot	SQLite (WAL)	A few trading days	What the live desk reads. Low latency, transactional, small working set.
Lake	Parquet, zstd-compressed	Never pruned	The permanent analytical record. Columnar, partitioned, cheap to scan.
Archive	Parquet, frozen	Permanent	Prior-generation capture, retained and still queryable.
Off-site	Object storage (4 buckets)	Permanent	Disaster recovery; encrypted for state, plaintext columnar for tape so it is queryable in place.

Parquet, exactly how it is used

- **Partitioning:** one file per `stream / instrument / day`. Currently 163 partitions, 702 MB. A query for one instrument-day touches exactly one file.
- **Written by a nightly mirror job** that exports completed days out of the hot store using the analytical engine's native `COPY ... (FORMAT parquet, COMPRESSION zstd)`. No heavyweight dependency is required.
- **Verified before it counts.** The mirror re-reads each written file and compares row counts against the source. A partition is only recorded as exported once that check passes.
- **Interlocked with retention.** The prune job is forbidden from deleting any day the mirror has not both exported and row-count verified. Data cannot be lost by a race between the two jobs.
- **Queryable in place**, locally or directly from object storage, with no import step.

Analytical engine (DuckDB)

- Used as an **embedded OLAP layer** — no server, no cluster, no separate infrastructure.
- **Attaches the hot SQLite stores read-only** and reads Parquet natively, so a single query can span the live tape and years of archive in one statement.
- A dedicated module presents all tiers as **one logical connection**, resolving overlaps by day boundary rather than blind union, and explicitly reporting coverage gaps so a study cannot silently assume contiguity across a seam.
- Handles the analytical workload: aggregations over tens of millions of rows, window functions, re-sampling, joins across tiers.

SQLite — still used, deliberately

- **Hot capture** (ticks, quotes, bars, book) — high-frequency append workload.
- **Trade store** — the P&L record of truth, with executions, orders and reconciliation state.
- **Research stores** — one per instrument, holding simulated decisions and their independently-scored results.

- Chosen because it is transactional, zero-administration, single-file, trivially backed up, and directly attachable by the analytical engine. The combination of SQLite for writes and DuckDB for analytics is the core of the storage design.

What is actually stored

Dataset	Rows held	Notes
Order-book records	55.1M	Top-of-book, event-driven
L2 depth snapshots	11.2M	10 levels each side, sub-second cadence, multi-instrument
Quotes	6.5M	
Trade ticks	5.3M	Used for honest re-pricing
Bars	567k	Sub-minute, aggregated upward on read
Simulated decisions	10,807	100% independently re-priced
Live trades, fills, orders	Full history	Reconciled against broker executions

Retention policy

- **Hot tier:** a small rolling window of trading days — bounded so the live desk's working set stays fast.
- **Lake:** never pruned. This is the permanent record.
- **Research stores:** never pruned. A simulated decision from any date remains queryable.
- **Off-site:** continuous; state hourly, tape nightly.
- Retention is **enforced by an interlock, not by a cron ordering assumption** — see the mirror/prune interlock above.

VII. Research engine

How hypotheses are generated

Four distinct sources, all of which end in the same registry:

1. **Structural observation** — recurring behaviour identified in the captured tape by exploratory analysis over the lake.
2. **Failure analysis** — decisions that lost, and decisions that nearly worked, are examined for the specific mechanism that failed. Each mechanism becomes a candidate variant.
3. **Report findings** — the weekly pipeline explicitly outputs a proposed slate of next experiments with the evidence attached.
4. **Parameter geography** — sweeps that map how a result behaves across a parameter surface, used to distinguish a genuine plateau from a fitted spike.

How experiments are defined

An experiment is a declarative record specifying its decision family, its parameters, its exit policy, its instrument, and any regime constraints. Adding one is a single declaration; the harness supplies execution, scoring, storage and presentation. Crucially:

- **Every experiment receives a permanent numeric identifier** from a registry, allocated once and never reissued — a retired experiment keeps its number forever. This is what makes "how is experiment 23 doing?" a well-formed question across months.
- **Constraints must be declared, not assumed.** Cooldowns, time caps and regime floors are enforced centrally by the harness so that a new experiment cannot silently omit the constraint its research ran under.

This last point is hard-won. Two separate incidents occurred where an experiment shipped without a constraint its own research had used — producing forward numbers that could not be compared to the study that justified it. The harness now enforces declared constraints centrally, and tests pin them.

How experiments are run

Candidates execute as **shadow arms**: the full decision path, fed the live tape, in parallel with the production desk, placing no orders. Dozens run concurrently at negligible cost. Because they consume the same data through the same code path as the live desk, a shadow result and a live result are comparable in a way that a separate backtester's output is not.

How results are stored and reproduced

- Each simulated decision is a row: instrument, direction, entry, exit, reason, timestamps, the volatility state at entry, and the exit parameters needed to replay it.

- A separate table holds the **independently re-priced result** and its fill status. The two are kept apart deliberately so an optimistic figure can never be mistaken for the honest one.
- Reproduction is possible because the stored row carries the parameters required to replay the exit against the captured tick data — the re-pricing engine does exactly this, and can be re-run at any time.
- Configuration changes to live behaviour are journalled, and live-behaviour files are version-controlled with an automated check that they are both committed and pushed off-machine.

How hypotheses are compared

Classification precedes ranking. The system requires every arm to be classified — control, A/B half, factorial cell, or slow-firing — before any performance comparison. This exists because a naive profit-sort once placed a deliberate control arm at the top of a kill list. That control is the only evidence that the live configuration is worth having; deleting it would have destroyed the baseline while appearing to be housekeeping.

- **Paired design.** Variants are armed in matched pairs differing in exactly one property, so an effect can be attributed to that property rather than to the cell it happened to occupy.
- **Controls are expected to lose** and are protected from retirement on that basis.
- **Judgement is on expectancy and robustness, not win rate.**

VIII. Backtesting and walk-forward

Architecture

Two complementary engines:

1. **Forward simulation** (the shadow desks) — the primary method. Out-of-sample by construction, because the data has not happened yet.
2. **Historical replay** — a race engine that replays a candidate against captured tick data, deciding for each position whether the stop or the target was reached first, in order, at tick resolution.

Inputs and outputs

Inputs	Outputs
Captured ticks, quotes, bars and depth from the lake; a candidate definition; explicit position, cooldown and hold constraints	Per-decision records with entry, exit, reason, hold time, favourable and adverse excursion; aggregate expectancy, hit rate, profit factor; robustness cuts

Validation methodology and overfitting controls

The system's stated methodological position is that **a backtest result is a hypothesis, not a finding**. The following are implemented rather than aspirational:

- **Tick-resolution race, never bar approximation.** The engine determines whether the stop or the target came first at tick level. A method that counts how often price reached a target while ignoring whether the stop came first was found to inflate an apparent hit rate dramatically and to have shipped a losing configuration; that method is banned.
- **Two independent scoring paths.** An optimistic bar-price figure and an honest tick-re-priced figure are computed separately. On one identical population the two disagreed in sign for half the arms. Only the honest one may be used for judgement.
- **Leave-one-out robustness.** Results are recomputed with the best day removed, and with the best three removed. A result carried by one session is visible immediately.
- **Both-halves test.** A result must hold in both halves of its window, not only in aggregate.
- **Placebo arms.** Candidates are run against randomised or sign-flipped variants; a signal that also "works" on the placebo is rejected.
- **Parameter-surface inspection.** Neighbouring parameter cells must also be positive. An isolated peak is treated as a fit, not an edge.
- **Look-ahead auditing.** Time-alignment defects are actively hunted — three separate look-ahead bugs were found inside the system's own research harnesses, each of which would have manufactured an edge that did not exist. Detection of these is now part of the weekly process.
- **Cost re-derivation.** Transaction cost is re-derived from mechanism rather than copied from a report, after an incorrect cost constant was used to kill a viable candidate.

Out-of-sample and feedback

Forward shadow running is the out-of-sample test, and it is continuous rather than a one-off holdout. A candidate that survives historical validation is armed in shadow and accumulates genuine unseen data from that moment. Promotion requires a stated forward threshold — a minimum expectancy over a minimum number of forward decisions, explicitly including a regime the candidate was not fitted on. Results feed back into the registry, into the weekly report, and into the next round of hypotheses.

IX. Near-miss and failure analysis

What is recorded

Record	Content
Decision journal	Every signal considered, including those suppressed, and why
Suppression events	Whether a strategy wanted to act and was blocked, versus could not have acted at all — the distinction that separates a real missed opportunity from a free one
Excursion data	Maximum favourable and adverse movement per decision — how close a loser came to working, and how much a winner gave back
Exit reason	Categorised for every decision, enabling exit-mix analysis
Execution quality	Fills compared against the visible order book at the time, flagging any fill outside all visible liquidity
Reconciliation state	Book versus broker executions, with days that cannot be verified explicitly refused rather than scored clean
Decision audit ledger	A scored, human-readable record of past routing decisions and their outcomes, including the bad ones

How near misses become hypotheses

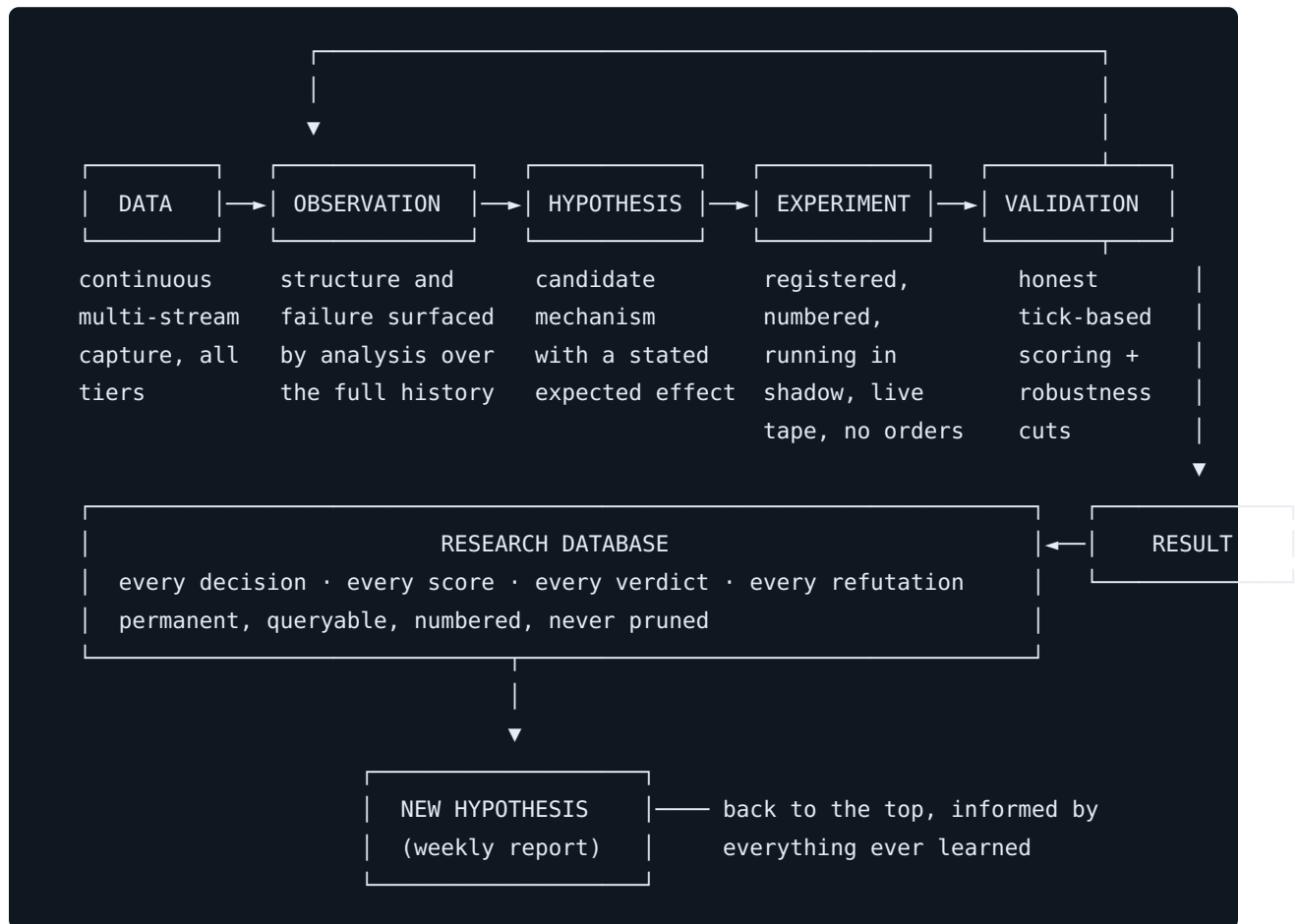
A decision that stopped out but had substantial favourable excursion first is not simply a loss — it is a timing or exit hypothesis. A decision that was suppressed by a filter and would have worked is a filter calibration hypothesis. Because the system records both the suppression and the counterfactual, these become measurable rather than anecdotal, and the weekly report surfaces them as proposed experiments.

How prior research is retained

- **Every lead ends in one of four states: live, shadow, parked, or refuted.** "Quietly abandoned" is not permitted. A parked lead retains its revival condition.
- **A refuted idea keeps a shadow arm running** where practical, so the refutation is continuously re-tested rather than frozen. A verdict resting on a small sample and no ongoing instrument is treated as an open question, not a closed one.
- **Null results are recorded as first-class findings** so they are not re-derived. Several are documented specifically to stop future work repeating them.
- An append-only session log, an architectural decision record, and a permanent experiment registry together mean the reasoning survives, not just the numbers.

X. The continuous-learning research loop

This is the mechanism the rest of the architecture exists to serve.



Stage by stage

Stage	What happens	What makes it durable
Data	Multi-stream capture runs continuously: ticks, quotes, bars, top-of-book, full L2 depth. Nothing is sampled away.	Three storage tiers with an interlock that makes data loss structurally impossible rather than merely unlikely.
Observation	The live layer maintains structured market state; analysis over the full history surfaces recurring structure and the mechanism behind losses and near-misses.	Suppressed signals and excursion data are recorded, so "what didn't happen" is as queryable as what did.
Hypothesis	A candidate mechanism is stated with an expected effect and the population it should apply to.	It is registered and permanently numbered before it runs. It cannot become anonymous.
Experiment	The candidate is armed as a shadow arm — full decision logic on the live tape, no orders — usually paired with a control or an A/B twin differing in one property.	Constraints are enforced centrally, so the forward run matches the study that justified it.
Validation	Every decision is independently re-priced on tick data. Robustness cuts, placebos, both-halves and parameter-neighbourhood checks are applied.	Two scoring paths kept deliberately separate; only the honest one may be used for judgement.
Result	A verdict: promote, park, or refute — with the evidence and the sample size attached.	"Quietly abandoned" is not an available state.
Database	The verdict and all underlying rows are written to permanent storage alongside every prior experiment.	Never pruned. Queryable years later. Numbered forever.
New hypothesis	The weekly report reads the accumulated record and proposes the next slate — including revisiting parked leads whose revival conditions are now met.	Each cycle starts from everything learned, not from zero.

The compounding property. Because every cycle adds rows and never removes them, the research base grows monotonically. A hypothesis proposed in month twelve is tested against twelve months of captured tape and evaluated in the context of every experiment already run. The system's analytical capability therefore increases with age — the opposite of a backtest script, whose value is fixed at the moment it is run and decays as the market moves on.

Stated honestly: the loop's hypothesis-generation stage is currently the least automated. Structure detection, failure analysis and report generation are automated; the judgement that turns an observation into a well-posed candidate is model-assisted and human-supervised. This is the most valuable place to invest for a commercial product, and it is also the place where over-claiming would be easiest.

XI. The weekly research report

Inputs

- The full live trade record with reconciled executions
- The complete shadow research book, honestly re-priced
- The routing decision log and its scored audit ledger
- The captured tape across all tiers, for fresh analysis
- Prior reports, so findings can be tracked across weeks

Architecture

A **21-phase pipeline**, one operating-system process per phase. Phases cover: live desk performance, the secondary desk, the shadow book, routing analysis, strategy rehabilitation, several independent greenfield research clusters, chart generation, then assembly, proofreading, revision and finalisation.

- **The artefact on disk is the checkpoint.** A re-run resumes rather than restarts.
- **One process per phase** keeps memory flat on a constrained host.
- **Judged on the artefact, never the exit code** — a phase that exits successfully having produced nothing is treated as a failure, because that exact silent failure occurred.
- **Budget-aware scheduling.** A preflight compares declared phase timeouts against the real time budget; each phase is capped at a fair share; a phase that would receive a useless sliver is skipped and logged rather than silently truncated. Greenfield research clusters rotate week to week so the full space is covered over time rather than only the phases that happen to run first.
- **The tail is reserved** — assembly, proofread, revision and finalisation get a protected allocation. Research can be lost; the write-up cannot.
- **A shared data contract** is injected into every phase so no phase can quietly use the wrong data window or the wrong cost assumption.

What the report contains and how it identifies discoveries

Per-strategy performance with robustness cuts; regime segmentation; exit-mix and excursion analysis; the shadow book as the explicit counterfactual for anything that was benched; greenfield research findings with placebo results; a register of instrument failures found in its own harnesses; and a closing section proposing the next slate of experiments with evidence and build cost per item.

A "discovery" is only reported with the sample size, the robustness cuts, and an explicit statement of what would refute it. The pipeline includes an adversarial revision pass whose job is to attack the draft's own conclusions before publication.

Can it operate indefinitely as the database grows?

Structurally yes, with one caveat. The columnar lake is partitioned by day and by instrument, so per-query cost scales with the window examined rather than with total

history — a one-week analysis costs the same in year five as in month one. Phases are independent processes with capped memory, so a larger corpus does not raise peak memory.

The caveat is that any phase deliberately scanning all history grows linearly. The mitigation already in place is the fair-share budget with explicit logging of what was skipped; the productised answer is pre-aggregated summary tables per period, which the architecture supports but which is not yet built.

XII. AI integration

Where the model is used today

Use	Cadence	Authority
Regime routing — which strategy families may open new positions	Every 5 min, headless	Writes one switch file. Cannot place, modify or cancel an order.
Session watch — dissect an escalation and decide whether a human is needed	Hourly in market hours	Read-only. Explicitly forbidden from changing desk state.
Health and audit reviews	Several times daily	Read-only reporting.
Weekly research report	Weekly, 21 phases	Produces documents; no execution authority.
Verification passes	On demand	Adversarial review of the system's own findings.

What the model receives and produces

Receives: deterministically-computed context — market state summaries, per-strategy performance, health status, position and reconciliation state, and an explicit rules prompt. It reads prepared facts; it does not roam.

Produces: a structured decision plus a written rationale. The rationale is logged in full, so every automated decision has a durable, human-readable justification attached — a property most automated systems lack entirely.

Deterministic versus generated

The boundary is strict and architectural. Everything that touches money is deterministic code: order placement, position management, stops, flatten logic, reconciliation, safety. The model's entire authority is a permission layer — which strategy families may open new positions — expressed as a single file. Any model error or malformed output results in no change, and the system continues on its previous state, benching-only. It cannot invent a trade.

Could it support other LLMs?

Yes, with modest work. Integration is via a command-line invocation returning structured output, with fail-safe parsing. Swapping providers means replacing that invocation and re-tuning prompts. The prompts themselves are external Markdown files, not embedded strings — deliberately, so they can be edited and versioned without code changes. Nothing depends on provider-specific APIs. For a commercial product this would be abstracted behind a provider interface; the seams are already in the right places.

XIII. Dashboard and user experience

Surface	What the user sees	Maturity
Desk view	Live price, position, day P&L, per-strategy performance, blotter, equity curve, regime indicators	Production
Research board	Every registered experiment with permanent number, friendly name, family tag, day/week/all-time honest P&L, and open-position state	Production
Secondary-instrument board	Dedicated page per research desk, with cost and methodology caveats printed on the page itself	Production
Reports	Weekly dossiers, browsable and downloadable as PDF	Production
Analytics cube	Multi-dimensional slice of trade outcomes	Working
Mobile control	Messaging-based status and control channel with de-duplicated alerting	Production

The dashboard is **read-only by design** — no order entry, no writes. A presentation fault cannot become a trading fault. Notable detail: the research board prints its own methodology caveats (cost basis, data-source divergence) directly on the page, so a number cannot be read out of context.

XIV. Broker and data integration

- **Gateway:** a containerised broker gateway holding one authenticated session. Currently Interactive Brokers.
- **Session isolation:** each desk connects with its own client identity. Cross-desk interference is structurally prevented and continuously verified.
- **Market data:** streaming trades, quotes and aggregated bars, republished on the internal bus.
- **Depth:** a separate service records 10-level book snapshots at sub-second cadence per instrument.
- **Historical:** broker historical data can be pulled for instruments predating local capture, with request pacing respected and a preflight that refuses to run while the desk holds a position — the research pull must never compete with trading for the session.
- **Account integration:** positions, orders and executions are read continuously. Broker state is treated as the sole truth; internal books are treated as intent and are reconciled against executions.

Required from the customer: a brokerage account with market-data entitlements for the instruments studied, credentials for the gateway, and a host meeting the deployment requirements below. Research-only operation needs data entitlements but no trading permissions.

XV. Deployment, scalability, security, limitations

Deployment

Element	Current state
Host	Single Linux VPS. Modest: the production system runs in 7.5 GB RAM.
Process management	systemd — 63 units with dependencies, restart policies, memory limits and OOM priorities
Containerisation	Broker gateway only; application runs natively in a Python virtual environment
Dependencies	Python 3.13, DuckDB, pandas, a broker client library, an object-storage sync tool, a PDF renderer. No database server, no message broker, no cluster.
Web	Reverse proxy with TLS in front of the read-only application server
Setup	Manual. Unit files and prompts are version-controlled; installation is not automated.

Scalability

- **Instruments:** the bus, capture and storage are already multi-instrument, and per-instrument research desks with isolated stores are an established pattern. Adding an instrument is largely configuration. The practical limit is market-data bandwidth and host memory, not architecture.
- **Trades and experiments:** negligible. Tens of thousands of research rows are trivial for the analytical layer.
- **Historical data:** the columnar lake is the scaling story. At 10× (~7 GB, ~1,600 partitions) nothing changes — partition pruning means query cost tracks the window examined. At 100× (~70 GB) the lake still functions locally, and queries can run directly against object storage; hot-tier retention and off-site sync windows would need attention, and pre-aggregated period summaries become worthwhile.
- **The real constraint is host memory during the weekly report**, already mitigated by one-process-per-phase and explicit memory limits.

Security

Area	Current	Product requirement
Broker credentials	Held by the containerised gateway	Adequate pattern; needs documented rotation
Object-storage keys	Sync-tool configuration on host	Move to a secrets manager
Model credentials	Session-based OAuth, requiring periodic interactive re-authentication	Blocking for unattended commercial operation — needs long-lived service credentials
Backups	State encrypted; tape stored as plaintext columnar so it stays queryable in place	Deliberate trade-off; must be a documented customer choice
Data isolation	Single-tenant by construction — one host, one account, one dataset	Multi-tenant isolation is unbuilt and would be the largest single piece of work
Dashboard	Read-only; no order entry; no write paths	Needs authentication and per-customer access control

Current limitations and technical debt

Stated plainly, because a buyer will find these anyway.

- **Absolute paths are hard-coded** throughout. Installation-root and data-root abstraction is a prerequisite for distribution.
- **Single-tenant.** No user model, no per-customer isolation, no authentication on the dashboard.
- **Instrument specifics appear in code** — contract multipliers, cost constants, session clocks. These need to become an instrument registry.
- **Operational knowledge lives in Markdown**, not in the software. The system is unusually well documented, but that documentation is currently a human-facing operating manual rather than product behaviour.
- **Session-based model authentication** requires periodic interactive login — the single largest obstacle to unattended commercial operation.
- **Retired-generation code** remains present alongside live code; a customer distribution needs it removed.
- **Manual installation.** No installer, no migrations, no upgrade path.
- **Prompts are tuned to one operator's judgement** and encode this desk's history. They generalise poorly without abstraction.
- **Alerting is single-channel** and assumes one operator.

Commercialisation requirements

Reusable as-is	Needs abstraction	Needs building
Three-tier storage model · Parquet lake and interlock · analytical query layer · shadow research harness · honest re-pricing engine · experiment registry · validation methodology · report pipeline architecture · reconciliation and safety layers · self-instrumentation patterns	Paths and configuration · instrument definitions · broker interface · model-provider interface · prompt templates · alerting channels	Installer and migrations · authentication and multi-tenancy · secrets management · long-lived model credentials · instrument onboarding UI · experiment-definition UI · customer documentation · licensing and update mechanism

Assessment: the research machine — storage, harness, scoring, validation, registry, reporting — is the durable asset and is production-grade. The product wrapper — installation, multi-tenancy, configuration, authentication, documentation — is largely unbuilt. That is a favourable ratio: the hard, subtle, expensive part is done and battle-tested; what remains is well-understood engineering.

XVI. Capability inventory

Maturity: **Production** = running unattended and depended upon · **Working** = functional, needs hardening · **Alpha** = exists, not ready to sell

Capability	What it does	Why it matters	Customer benefit	Maturity
Persistent research database	Stores every simulated decision, score and verdict in an analytically queryable historical layer that is never pruned.	Research isn't lost when an experiment ends.	The system accumulates institutional-style research memory over time; year two is more capable than year one.	Production
Three-tier storage	Hot transactional tier for live reads, permanent columnar lake for analysis, off-site replication — with an interlock preventing deletion of unverified data.	Most systems must choose between fast live access and deep history. This has both.	Years of tick-level history without a database server, a cluster, or a data engineer.	Production
Parallel shadow research	Dozens of candidate strategies execute full decision logic against the live tape simultaneously, placing no orders.	Forward testing is out-of-sample by construction — the data hasn't happened yet.	Test many ideas at once on live data with zero capital at risk and zero incremental cost.	Production
Honest re-pricing engine	Independently re-scores every simulated decision against captured tick data rather than bar prices.	Two scoring paths on one identical population disagreed in sign for half the arms tested.	Numbers you can act on, instead of a backtest that flatters itself.	Production
Permanent experiment registry	Allocates an immutable number to every experiment; retired ones keep their number forever.	"How is experiment 23 doing?" stays a well-formed question across months.	Research is referable and auditable over time, like lab notebook entries.	Production
Controls and paired design	Deliberate control arms and matched A/B twins differing in exactly one property, protected from performance-based deletion.	Without a control you cannot attribute an effect; a naive profit-sort once put the control top of a kill list.	Findings are attributable to a cause rather than to a coincidence.	Production
Anti-overfitting battery	Leave-one-out and leave-three-out cuts, both-halves tests, placebo arms, parameter-neighbourhood	Nearly every retail backtest is overfitted and has no way to detect it.	You find out an edge is fictional before risking money, not after.	Production

	inspection, look-ahead auditing.			
Near-miss and suppression analysis	Records excursion data and whether a strategy wanted to act and was blocked, versus could not have acted at all.	The difference between a real missed opportunity and a free one is invisible to most systems.	Losses and non-events become a hypothesis pipeline instead of noise.	Production
Automated weekly research report	21-phase pipeline producing a structured dossier with findings, robustness cuts, adversarial review and a proposed next slate.	Converts accumulated data into decisions on a fixed cadence, without a human analyst.	A quantitative research team's weekly output, generated unattended.	Production
Resumable long-running pipelines	Artefact-on-disk checkpointing, one process per phase, fair-share time budgeting, explicit logging of anything skipped.	Long analytical jobs fail; the question is whether they fail honestly and resume.	Multi-hour research completes reliably on modest hardware, and never silently under-delivers.	Production
Multi-stream market capture	Ticks, quotes, bars, top-of-book and full 10-level depth, continuously, multi-instrument.	Microstructure research is impossible without depth data, and it cannot be bought retroactively.	Every day of operation permanently increases the research surface available to you.	Production
Unified query across all history	One logical connection spanning live tier, lake and archive, resolving overlaps by boundary and reporting coverage gaps explicitly.	An answer that scans one tier is wrong, and being wrong quietly is the worst outcome.	Ask a question of your entire history in one statement, and be told where the holes are.	Production
Execution-quality measurement	Compares every fill against the visible order book at the time and flags fills outside all visible liquidity.	Book and broker can agree on a bad price — agreement is not quality.	Slippage and poor fills are measured and costed rather than assumed.	Production
Multi-desk reconciliation	Continuously enforces that broker position equals the sum of every desk's claim; halts on	On a netted account no strategy can read its own position from the venue.	Run several strategies on one account without them corrupting each other's state.	Production

breach and never self-releases.

Self-instrumentation	Health checks designed against the failure of reporting healthy about something never actually checked; alarms name exactly what they verified.	Silent no-ops are the dominant failure mode of automated systems.	When the dashboard is green, it means something specific — and you know what.	Production
De-duplicated alerting	Alerts keyed on situation rather than on changing values, cleared on resolve, failing open on any error.	An alarm that repeats a correct message forever trains you to ignore the channel.	Notifications you still read after six months.	Production
Model-assisted regime routing	An LLM decides which strategy families may open positions, writing a single permission file and logging a full rationale.	Judgement about regime is where models add value; execution is where they do not.	Adaptive behaviour with a written justification for every decision — and no ability to invent a trade.	Working
Deterministic/AI separation	All money-touching paths are deterministic; the model's entire authority is a permission layer. Any failure results in no change.	The critical safety question for any LLM trading product.	Model benefits without model risk on the order path.	Production
Lead lifecycle discipline	Every idea ends live, shadow, parked or refuted — with revival conditions recorded and refuted ideas often left running.	"Quietly abandoned" is how research is silently lost.	Nothing is forgotten, and no ground is re-litigated from scratch.	Production
Read-only analytics surface	Dashboards for desk, research board, reports and analytics cube, with methodology caveats printed alongside the numbers.	A presentation fault must never become a trading fault.	See everything, break nothing.	Production
Configuration journalling	Live-behaviour changes are journalled and	An uncommitted live change cannot	Full auditability of what the system was	Production

	version-controlled, with an automated check that they are committed and pushed off-machine.	be diffed, reverted or attributed.	doing on any past date.	
Historical broker backfill	Paced retrieval of vendor history for instruments predating local capture, refusing to run while the desk holds a position.	New instruments otherwise start with no history at all.	Onboard an instrument with years of history rather than waiting years.	Working
Multi-instrument research desks	Per-instrument research desks with isolated stores, separate cost models and independent boards.	Instruments differ in cost, multiplier and microstructure; sharing one store corrupts all three.	Research several markets in parallel without cross-contamination.	Working
Automated installation	—	—	—	Not built
Multi-tenancy & auth	—	—	—	Not built

XVII. The skeptic's ten

Written in the voice of a professional systematic trader evaluating GAZBOT, based only on what is actually in the codebase — not on what would sound impressive.

I have looked at a lot of retail trading software. Most of it is a strategy with a chart attached. Here is what is genuinely unusual here, and I have deliberately excluded the things that merely sound good.

1. It re-prices its own simulations against tick data, and keeps the optimistic number separately so it cannot be used by accident

Nearly every backtester I have seen fills at the bar. This one replays each simulated position against captured ticks and asks which came first, the stop or the target. It stores the optimistic figure and the honest figure in separate tables. On one identical population of several hundred decisions, the two disagreed **in sign** for half the strategies. Most products would have shipped the flattering number and never known.

2. Rejected strategies keep running

Everywhere else, a rejected idea is deleted. Here a refuted idea is often left running in simulation so the refutation is continuously re-tested. During review, a strategy benched eighteen days earlier on a sample of eleven decisions was identified as having no ongoing instrument — and the response was to arm a shadow twin so the verdict gets priced continuously. That is a research culture encoded as software, not a feature list.

3. It runs deliberate control arms that are supposed to lose, and protects them from deletion

The single most professional thing in this system. There are control arms carrying five-figure losses whose entire purpose is to be the baseline. The codebase explicitly records an incident where a profit-ranking put the control at the top of a kill list, and the rule that emerged — classify before you rank — is now enforced. I have worked at firms that did not do this.

4. Its self-monitoring is designed against a named failure mode, not against "is the process alive"

The recurring theme is: an instrument that reports healthy about something it never checks. Concrete instances found and fixed include a monitor that verified a job was committed but never that it had left the machine; a research desk that was "active", consuming CPU, and had never once been capable of firing; and staleness alarms measured on wall-clock across a weekend when the market was shut. Each produced a specific check. This is a system that has been embarrassed by reality and learned.

5. Look-ahead bugs are hunted inside its own research harnesses, and found

Three separate time-alignment defects were caught in its own tooling — a five-second look-ahead that manufactured an entire regime-specific edge, a leak that reversed a strong statistical result once removed, and an integer-division error that silently read five-second bars as one-minute bars for hours. Each is documented with what it was worth. Anyone can claim they avoid look-ahead; this is the only retail-scale system I have seen that treats finding them as a standing weekly job.

6. The storage design is genuinely good engineering, not a database with a folder next to it

Hot transactional tier for the live path, permanent columnar lake for analysis, off-site replication, and — the part I would actually steal — an **interlock forbidding the prune job from deleting any day the mirror has not exported and row-count verified**. That is the difference between "we have backups" and "data loss is structurally impossible". Tens of millions of rows, tick-level history, and no database server or cluster anywhere.

7. The LLM cannot place a trade, and that boundary is architectural rather than a promise

The model's total authority is a permission file naming which strategy families may open positions. Every money-touching path is deterministic. A malformed model response results in no change and the system continues benching-only. Given how many "AI trading bots" hand a language model an order API, having the boundary drawn in exactly the right place — judgement in, execution out — is the most credible design decision here.

8. Every automated decision carries a written rationale, permanently

The routing layer logs a full paragraph of reasoning for every five-minute decision, and there is a scored audit ledger grading past decisions — including the bad ones, with the lesson extracted. I can go back to any afternoon and read why the system did what it did. I cannot do that with any commercial product I have used, and I frequently cannot do it with my own code.

9. Experiments have permanent immutable identifiers

Small, and it changes everything. Numbers are allocated once and never reissued; a retired experiment keeps its number. It means "how is 23 doing?" survives across months. The codebase also records the failure of a dashboard that numbered experiments by array position and drifted until most numbers were wrong — and the fix was to make the display read the registry. That is a team that understands that two surfaces disagreeing is the actual enemy.

10. It knows, and records, the difference between "no setup" and "broken"

This is the one I would pay for. Most systems cannot distinguish a strategy that found no opportunity from a strategy that is silently dead. This one records suppressed signals, whether a strategy wanted to act and was blocked versus could not have acted at all, and now publishes open simulated positions so a blocked strategy is visibly blocked. It found a research desk that had produced nothing for three days while appearing perfectly healthy — and the response was both to fix it and to build the check that would have caught it on day one.

What would still make me hesitate

In fairness, and because a buyer will ask:

- **The forward sample is young.** The methodology is excellent; the accumulated evidence is weeks, not years. The machine is more proven than any individual finding it has produced.
- **It is single-tenant and hand-installed.** Nothing about the research core prevents productisation, but the wrapper does not exist yet.
- **Unattended operation depends on session-based model credentials** that need periodic interactive renewal. That must be solved before anyone sells this.
- **The hypothesis-generation step is the least automated part of the loop** — which is precisely the part the marketing will be most tempted to overstate.

Summary judgement. As a trading bot this would be one of many. As a **research instrument** it is unusual, and the unusual parts are the ones that are hard to copy: honest scoring, controls, permanent research memory, and self-instrumentation built by being wrong and noticing. The product is not the strategies. The product is the machine that evaluates them.

Prepared 18 August 2026 from direct inspection of the running system. All figures measured, not estimated. **No trading logic, entry or exit conditions, thresholds or parameter values are disclosed in this document.**